Swarm and Evolutionary Computation 78 (2023) 101294

Contents lists available at ScienceDirect

Swarm and Evolutionary Computation

journal homepage: www.elsevier.com/locate/swevo

MjSO: A modified differential evolution with a probability selection mechanism and a directed mutation strategy

Yintong Li [a], Tong Han [a], Xiaofei Wang [b,*], Huan Zhou [a], Shangqin Tang [a], Changqiang Huang [a], Bo Han [c]

[a] Aviation Engineering School, Air Force Engineering University, Xi'an 710038, China
[b] Beijing Blue Sky Innovation Center for Frontier Science, Beijing 100085, China
[c] 93525 Unit, Shigatse 857000, China

ARTICLE INFO

Keywords:
Numerical optimization
Differential evolution
L-SHADE
CEC 2018 test suite

ABSTRACT

To further enhance the performance of jSO, the most representative variant of differential evolution, a modified differential devolution with a probability selection mechanism and a directed mutation strategy named MjSO is developed in this study. There are three main modifications for jSO. Firstly, a novel probability selection mechanism is utilized in mutation process to fully take advantage of dominant individuals. Specifically, the better dominant individual has a higher probability of being selected. Then, a directed mutation strategy is adopted to ensure the evolution towards a better direction during the optimization. Moreover, the archive scheme is no longer adopted in MjSO to avoid the excessive use of historically dominant individuals. MjSO is evaluated using the CEC 2018 test suite in comparison with six state-of-the-art DE variants, called jSO, mL-SHADE, ELSHADE-SPACMA, EB-LSHADE, iL-SHADE, and MadDE. The numerical experimental results certify that MjSO outperforms the six comparison algorithms. Moreover, MjSO significantly enhances the performance of jSO while reducing its time complexity. In conclusion, the probability selection mechanism and directed mutation strategy proposed in this study efficiently enhance the performance of jSO, and MjSO is a promising algorithm.

1. Introduction

Modern meta-heuristic algorithms have been found to be more effective than algorithms based on gradient information for generating optimal or near-optimal solutions for non-separable rough optimization [1]. Success-history-based adaptive differential evolution using linear population size reduction (L-SHADE) [2], one of the most competitive variants of differential evolution (DE) [3], has been claimed to be an excellent algorithm for solving optimization problems because it is the winner in the 2014 IEEE Congress on Evolutionary Computation. Thus, L-SHADE is used to solve many real-world optimization problems [4]. To further exploit L-SHADE, many improvements have been proposed for its drawbacks since it was proposed in 2014. Most existing variants of L-SHADE could be generalized into four categories: adaptive technology for parameter control, modification for mutation strategy, modification for selection scheme, and hybridization with other evolutionary algorithms.

The first category is the adaptive technology for parameter control. In Ref. [5], LSHADE-EpSin was proposed by introducing an ensemble sinusoidal approach, that was used to automatically adapt the values of the scaling factor, to balance the exploitation and the exploration. In Ref. [6], iL-SHADE was proposed by modifying the memory update mechanism and initial historical memory values in the $\boldsymbol{M}_{CR}$ to improve the adaptive ability and robustness of L-SHADE. In iL-SHADE, $\boldsymbol{M}_{CR}$ and $\boldsymbol{M}_F$ of the current generation are weighted used for updating for the next generation to fully use the domain information. In addition, higher CR and lower F values were propagated early in the evolutionary process to enrich the diversity of the population. In Ref. [7], a variant of the iL-SHADE algorithm called jSO was proposed, in which a new parameter F_{w}, based on the scaling factor F, was introduced to the mutation strategy to tune the influence of $\boldsymbol{x}_{\mathrm{pbest}}$ in different stages of the evolutionary process to balance exploitation and exploration. In Ref. [8], a novel adaptive parameter control approach based on a reinforcement learning algorithm called policy gradient was introduced into DE to propose a learning adaptive differential evolution algorithm (LDE). In Ref. [9], a variant of L-SHADE named MadDE was proposed by

* Corresponding author.
E-mail address: wxf825421673@163.com (X. Wang).

https://doi.org/10.1016/j.swevo.2023.101294
Received 5 October 2022; Received in revised form 5 February 2023; Accepted 15 March 2023
Available online 16 March 2023
2210-6502/© 2023 Elsevier B.V. All rights reserved.

leveraging the power of the multiple adaptation strategy with respect to the control parameters and search mechanisms.

The second category is the modification for mutation strategy. In Refs. [10,11], ELSHADE-SPACMA was proposed by introducing a dynamic p-value and directed mutation strategy into LSHADE-SPACMA to enhance exploration and exploitation. In Ref. [12], a modified version of LSHADE with a rank-based selective pressure strategy (LSHADE-RSP) was proposed. The main difference between LSHADE-RSP and LSHADE is that the rank-based mutation scheme (*current-to-pbest/r*) is used in LSHADE-RSP; thus, the second part containing two vectors ($\boldsymbol{x}_{r1}$ and $\boldsymbol{x}_{r2}$) are selected according to the rank selection typically used in genetic algorithms [13] instead of being selected randomly. In Ref. [14], to further increase the exploration of LSHADE-RSP, the long-tailed property of the Cauchy distribution was used in the improved LSHADE-RSP (iL-SHADE-RSP), which perturbed the target vector instead of the mutant vector based on the jumping rate to generate a trial vector with high diversity. In Ref. [15], a population-wide inertia term (PWI) was added to the mutation strategies of four L-SHADE variants to improve their performance. The PWI term denotes the average moving direction and step of previous generation of successful individuals, that could guide individuals to evolve in a better direction. In Ref. [16], mL-SHADE was proposed by removing the terminal value, adding a polynomial mutation, and proposing a memory perturbation mechanism to enhance the performance of L-SHADE. In Ref. [17], Yi et al. proposed a novel differential evolution with ensemble populations, in which two sets of mutation and crossover operators are utilized to generate offspring to balance exploitation and exploration. In Ref. [18], two new mutation strategies mimicking the gradient decent behavior to direct the search toward better solutions, named *DE/current-to-ord_best/1* and *DE/current-to-ord_pbest/1*, were proposed to tune the exploration and the exploitation of DE. Moreover, Mohamed et al. [19] made a comprehensive analysis of the main existing mutation strategies for DE and gave some suggestions and guidelines for experienced practitioners to develop more effective DE variants.

The third category is the modification for selection scheme. In Ref. [20], LSHADE-EpSin was further improved by making two modifications. In one part, instead of randomly selecting a sinusoidal wave, an adaptive selection scheme based on the success of a collection of previous sinusoidal methods was employed. In another part, the Euclidean neighborhood covariance matrix is utilized in crossover operator to structure an appropriate coordinate system to improves the ability for tackling non-separable problems. The enhanced version of the LSHADE-EpSin was named LSHADE-cnEpSin, that was the winner of CEC 2017. In Ref. [21], a orthogonal array-based initialization, a conservative selection scheme, and an ensemble strategy for parameter adaptation techniques were incorporated in DE to develop a new variant of DE, named OLSHADE-CS. In Ref. [22], a new selection operator was adopted in DE to escape the local optimal value once the search process is in stagnation, in which the best, second-best vector of all the discarded trial vectors, and a random candidate chosen from all the successfully updated solutions may survive to the next generation. In Ref. [23], a selection operator based on the best discarded vector was utilized in DE to improve the ability to get rid of the local optimal value, in which the target vector will be replaced with the best vector of the target vector or a random vector chosen from the best discarded trial vectors for each individual if the search process is in stagnation.

Moreover, there are also some variants of L-SHADE by hybridizing it with other evolutionary algorithms. In Ref. [24], a hybrid algorithm called LSHADE-SPACMA was proposed by combining LSHADE-SPA (L-SHADE with semi-parameter adaptation for the scaling factor and crossover rate) and an improved version of Covariance Matrix Adaptation Evolutionary Strategy to fully exploit the advantages and characteristics of them. In Zhao et al. [25] proposed a hybrid cooperative differential evolution named jSO_CMA-ES_LBFGS by combining jSO and CMA-ES with the local search of Limited-Memory Broyden–Fletcher–Goldfarb–Shanno (LBFGS) mechanism to solve the complex problems.

The existing L-SHADE variants have improved its performance from different aspects. In particular, jSO significantly enhanced its performance in the basic framework of L-SHADE. Although jSO has shown competitive performance in many problems, there are still some defects, such as easily falling into local optima and not fully utilizing the dominant population, which limits its application scenarios. Thus, a novel modified jSO (MjSO) is proposed in this work to overcome above-mentioned defects for further improvement.

The remaining content of the paper is constructed as follows. Section 2 describes the principle of jSO. Section 3 introduces the model of MjSO. Section 4 discusses numerical experiments. Finally, conclusions of this paper and future directions are drawn in Section 5.

2. Principle of jSO

Recently, DE is still selected as a basis for developing novel evolution algorithms in the continuous optimization area [26–31], even though it was developed in 1995. And jSO [7] is regard as one of the most successful variants of DE over the past decades because it was the winner of the IEEE Competitions in Evolutionary Computation in 2017 (IEEE CEC 2017). The principle of jSO is presented as follows.

2.1. Initialization operator

The initial population is created using Eq. (1).

$$x_{i,j} = lb_j + r\cdot\left(ub_j - lb_j\right) \tag{1}$$

where, $i \in \{1, 2, 3\cdots, NP\}$ and $j \in \{1, 2, 3\cdots, D\}$ denote the index of an individual and the dimension index of an individual, respectively. NP is the size of population and D is the dimensionality. ub_j and lb_j are the upper and lower boundaries of the jth dimensional search space. r is a random number evenly distributed between 0 and 1. All entries in $\boldsymbol{M}_{CR}$ are initialized to 0.8, and all entries in $\boldsymbol{M}_F$ are initialized to 0.3. Moreover, the archive $\boldsymbol{A}$ is initialized to empty.

2.2. Mutation operator

The mutation operator "*DE/current-to-pBest-w/1*" is adopted to generate the mutated vector $\boldsymbol{v}_i$, and its model is described as follows:

$$\mathbf{v}_i = \mathbf{x}_i + Fw_i\cdot\left(\mathbf{x}_{pbest} - \mathbf{x}_i\right) + F_i\cdot\left(\mathbf{x}_{r1} - \mathbf{x}_{r2}\right) \tag{2}$$

$$F_i = \mathrm{Cauchy}\left(M_{F,r3}, 0.1\right) \tag{3}$$

$$Fw_i = \begin{cases} 0.7\cdot F_i, & FEs < 0.2\cdot FEs_{\max} \\ 0.8\cdot F_i, & 0.2\cdot FEs_{\max} \le FEs < 0.4\cdot FEs_{\max} \\ 1.2\cdot F_i, & FEs \ge 0.4\cdot FEs_{\max} \end{cases} \tag{4}$$

$$N_{\mathrm{pbest}} = NP\cdot\left(p_{\min} + \left(p_{\max} - p_{\min}\right)\cdot\frac{FEs}{FEs_{\max}}\right) \tag{5}$$

where, $\boldsymbol{x}_{pbest}$ is randomly selected from N_{pbest} dominant individuals, $\boldsymbol{x}_{r1}$ is randomly selected from the current population and $\boldsymbol{x}_{r2}$ is randomly selected from the combination of current population and external archive [32]. $\boldsymbol{x}_i$, $\boldsymbol{x}_{r1}$, and $\boldsymbol{x}_{r2}$ are distinct individuals. F_i denotes the scaling factor and it obeys the Cauchy distribution, and $M_{F,\,r3}$ is an entry randomly selected from $\boldsymbol{M}_F$. It is noted that Eq. (3) is repeatedly implemented if F_i is not greater than 0 and F_i is set to 1 if F_i is greater than 1. FEs and $FEs_{\max}$ denote the number of current and maximum evaluations, respectively. Moreover, Eq. (6) will be adopted to correct $\boldsymbol{v}_i$ if it is beyond the boundaries of the search space.

$$v_{i,j} = \begin{cases} 0.5\cdot\left(lb_j + x_{i,j}\right), & v_{i,j} < lb_j \\ 0.5\cdot\left(ub_j + x_{i,j}\right), & v_{i,j} > ub_j \end{cases} \tag{6}$$

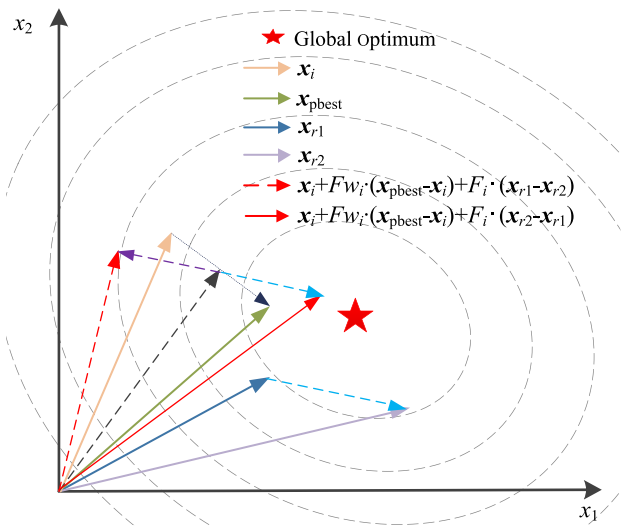


Fig. 1. Illustration in a two-dimensional search space of the directed mutation strategy.

2.3. Crossover operator

The binomial crossover strategy is adopted to create a trial vector u_i, and its model is described as follows:

$$u_{i,j} = \begin{cases} v_{i,j}, & r < CR_i \text{ or } j = j_{rand} \\ x_{i,j}, & \text{otherwise} \end{cases} \tag{7}$$

$$CR_i = \begin{cases} 0, & M_{CR,r3} = \perp \\ \text{Gaussian}\left(M_{CR,r3}, 0.1\right), & \text{otherwise} \end{cases} \tag{8}$$

where, j_{rand} is randomly selected from $\{1, 2, 3, \cdots, D\}$. CR_i denotes the crossover rate and it obeys the Gaussian distribution. It is noted that CR_i will be set to 1 if it is greater than 1 and CR_i will be set to 0 if it is less than 0. $M_{CR,\ r3}$ is an entry randomly selected from $\boldsymbol{M}_{CR}$, and $\perp$ denotes the terminal value.

2.4. Selection operator

The survivor for the next generation is selected using greedy mechanism as shown in Eq. (9).

$$\boldsymbol{x}_i = \begin{cases} \boldsymbol{u}_i, & f(\boldsymbol{u}_i) \le f(\boldsymbol{x}_i) \\ \boldsymbol{x}_i, & \text{otherwise} \end{cases} \tag{9}$$

where, f is the objective function. Moreover, $\boldsymbol{x}_i$ will survive to the next generation if it is better than $\boldsymbol{u}_i$. Otherwise, $\boldsymbol{u}_i$ will survive to the next generation and $\boldsymbol{x}_i$ will be preserved to the external archive $\boldsymbol{A}$. If archive $\boldsymbol{A}$ is full, a random individual contained in the archive is deleted to make room for the newly inserted individual.

2.5. Historical memory update operator

The successful CR_i and F_i are preserved in $\boldsymbol{S}_{\mathrm{CR}}$ and $\boldsymbol{S}_F$, respectively, for updating $\boldsymbol{M}_{CR}$ and $\boldsymbol{M}_F$. The memory update operator is described as follows:

$$M_{F,k} = \begin{cases} M_{F,k}, & S_F = \varnothing \\ 0.5\cdot\left(\text{mean}_{WL}(\boldsymbol{S}_F) + M_{F,k}\right), & \text{otherwise} \end{cases} \tag{10}$$

$$M_{CR,k} = \begin{cases} 0, & M_{CR,k} = \perp \\ M_{CR,k}, & \boldsymbol{S}_{\mathrm{CR}} = \varnothing \\ 0.5\cdot\left(\text{mean}_{WL}(\boldsymbol{S}_{CR}) + M_{CR,k}\right), & \text{otherwise} \end{cases} \tag{11}$$

$$\text{mean}_{WL}(\boldsymbol{S}) = \frac{\sum_{n=1}^{|S|}\omega_n\cdot S_n^2}{\sum_{n=1}^{|S|}\omega_n\cdot S_n} \tag{12}$$

$$\omega_n = \frac{|f(\boldsymbol{u}_n) - f(\boldsymbol{x}_n)|}{\sum_{n=1}^{|S|}|f(\boldsymbol{u}_n) - f(\boldsymbol{x}_n)|} \tag{13}$$

where, $\boldsymbol{S}$ refers to either S_{CR} or S_F in this paper. Only a pair of entries in $\boldsymbol{M}_{CR}$ and $\boldsymbol{M}_F$ are updated at each generation. If the index k is greater than the memory size H, it will be set to 1. Moreover, the entries with index H in $\boldsymbol{M}_{CR}$ and $\boldsymbol{M}_F$ always keep 0.9 during the optimization.

Algorithm 1: MjSO.

```
Begin:  // Initialization
        Initialize NP_init, NP_min, FEs, FEs_max, p_max, p_min, m, H, M_CR, M_F.
        Generate initial population using Eq. (1).
        Evaluate initial population and update FEs.
        // Main loop
        While FEs< FEs_max do
          For i=1 to NP do
            Calculate F_i, Fw_i, CR_i, and Pr_i as per Eq. (3), Eq. (4), Eq. (8), and Eq. (15).
            Generate mutated vector v_i using Eq. (17) or Eq. (18) according to the problem type.
            Correct v_i using Eq. (6).
            Generate u_i using Eq. (7).
          End for
          Evaluate all u and update FEs.
          For i=1 to NP do
            if f(u_i)≤f(x_i) then
              x_i = u_i.
            End if
            if f(u_i)<f(x_i) then
              S_CR ← CR_i ∪ S_CR.
              S_F ←F_i ∪ S_F
            End if
          End for
          Update M_F and M_CR using Eq. (10) and Eq. (11).
          Calculate NP using Eq. (14).
        End while
Return: Best solution.
```

Fig. 2. Pseudo-code of MjSO.

Table 1
Performance comparison of MjSO with different parameter m (D=30).

	F7		F8		F14		F17		F21		F24		F-Ranks
	Mean	SD	Mean	SD	Mean	SD	Mean	SD	Mean	SD	Mean	SD	
m=1	3.91E+01	1.62E+00	9.26E+00	1.70E+00	**2.06E+01**	4.77E+00	3.45E+01	6.85E+00	2.09E+02	1.79E+00	4.24E+02	2.64E+00	9.67
m=2	3.95E+01	1.48E+00	9.59E+00	2.31E+00	2.21E+01	7.92E-01	3.36E+01	7.14E+00	2.10E+02	1.41E+00	4.23E+02	2.07E+00	14.00
m=3	3.88E+01	1.18E+00	9.16E+00	1.82E+00	2.29E+01	1.88E+00	3.35E+01	5.38E+00	2.10E+02	1.80E+00	4.23E+02	2.30E+00	10.00
m=4	3.94E+01	1.78E+00	9.38E+00	1.99E+00	2.27E+01	1.08E+00	3.15E+01	8.19E+00	2.09E+02	1.68E+00	4.23E+02	2.27E+00	9.50
m=5	3.88E+01	1.74E+00	9.42E+00	1.66E+00	2.23E+01	2.51E+00	3.20E+01	5.89E+00	**2.09E+02**	1.72E+00	4.23E+02	2.15E+00	7.67
m=6	3.90E+01	1.24E+00	9.11E+00	1.95E+00	2.17E+01	2.73E+00	**3.04E+01**	7.83E+00	2.10E+02	2.03E+00	4.24E+02	2.06E+00	**6.67**
m=7	3.91E+01	1.70E+00	9.34E+00	1.56E+00	2.21E+01	1.26E+00	3.32E+01	6.80E+00	2.10E+02	1.89E+00	4.24E+02	2.26E+00	11.17
m=8	3.89E+01	1.71E+00	9.29E+00	1.35E+00	2.22E+01	1.42E+00	3.18E+01	8.01E+00	2.10E+02	2.51E+00	4.24E+02	1.63E+00	9.67
m=9	3.96E+01	1.33E+00	9.99E+00	2.03E+00	2.25E+01	1.69E+00	3.12E+01	7.04E+00	2.09E+02	1.33E+00	4.23E+02	2.45E+00	12.17
m=10	3.93E+01	1.53E+00	9.48E+00	1.70E+00	2.22E+01	1.11E+00	3.37E+01	6.47E+00	2.09E+02	1.73E+00	**4.23E+02**	2.24E+00	8.50
m=11	3.94E+01	1.81E+00	**8.73E+00**	1.75E+00	2.21E+01	1.63E+00	3.07E+01	6.77E+00	2.09E+02	2.24E+00	4.24E+02	2.01E+00	8.17
m=12	3.89E+01	1.51E+00	9.17E+00	1.82E+00	2.24E+01	1.61E+00	3.40E+01	6.87E+00	2.09E+02	1.82E+00	4.23E+02	1.83E+00	8.83
m=13	3.92E+01	2.33E+00	9.57E+00	1.43E+00	2.24E+01	1.65E+00	3.22E+01	6.06E+00	2.09E+02	2.03E+00	4.23E+02	2.91E+00	10.17
m=14	3.93E+01	1.33E+00	9.59E+00	1.67E+00	2.12E+01	3.47E+00	3.25E+01	6.17E+00	2.09E+02	1.57E+00	4.24E+02	2.26E+00	10.83
m=15	3.97E+01	1.56E+00	9.40E+00	1.58E+00	2.25E+01	1.71E+00	3.09E+01	5.40E+00	2.10E+02	2.64E+00	4.24E+02	1.46E+00	13.83
m=16	3.96E+01	1.72E+00	9.60E+00	2.05E+00	2.13E+01	5.30E+00	3.29E+01	7.28E+00	2.09E+02	1.76E+00	4.23E+02	2.85E+00	10.17
m=17	3.98E+01	1.51E+00	9.50E+00	2.08E+00	2.15E+01	2.45E+00	3.25E+01	6.55E+00	2.10E+02	1.55E+00	4.23E+02	2.19E+00	11.83
m=18	3.94E+01	1.63E+00	9.69E+00	1.86E+00	2.24E+01	1.59E+00	3.38E+01	4.87E+00	2.10E+02	1.64E+00	4.24E+02	2.42E+00	16.17
m=19	**3.87E+01**	1.47E+00	9.30E+00	2.06E+00	2.23E+01	1.19E+00	3.27E+01	5.28E+00	2.09E+02	1.71E+00	4.23E+02	1.89E+00	8.67
m=20	3.93E+01	1.53E+00	9.35E+00	2.06E+00	2.23E+01	1.43E+00	3.39E+01	7.45E+00	2.10E+02	1.36E+00	4.23E+02	2.36E+00	12.33

p-value derived from the Friedman test is 5.21E-01, and the *chi-square* is 18.02.

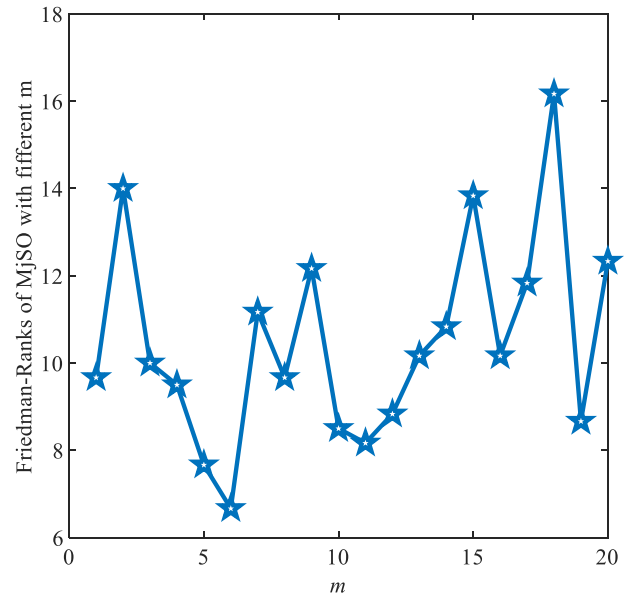


Fig. 3. Rankings of MjSO with different m settings obtained from the Friedman test.

2.6. Population size update operator

A linear function is adopted to dynamically reduce the population size during the optimization, and its model is described as follows:

$$NP = \text{round}\left[\frac{FEs}{FEs_{\max}}(N_{\min} - N_{\text{init}}) + N_{\text{init}}\right] \tag{14}$$

where, round () returns the nearest integer number. N_{min} is the minimum size of the population and N_{init} is the initial size of the population. The worst-ranking individuals included in the current population will be discarded once NP is updated.

3. Mathematical model of MjSO

The modifications for jSO mainly includes three parts. Firstly, a novel probability selection mechanism is utilized in mutation process to fully take advantage of dominant individuals. Specifically, the better dominant individual has a higher probability of being selected. Then, a directed mutation strategy is adopted to ensure the evolution towards a better direction during the optimization. Moreover, the archive scheme is no longer adopted in MjSO to avoid the excessive use of historically dominant individuals.

3.1. Probability selection mechanism

To fully take advantage of the dominant individuals, the N_{pbest} dominant individuals are given different selected probability according to their quality. A better individual will be given a greater selected probability, and vice versa. The probability selection mechanism can guide individuals to evolve in a better direction and accelerate the rate of convergence. Specifically, the N_{pbest} dominant individuals are ranked based on their quality from best to worst. The selected probability Pr_i for the ith individual, included in the top N_{pbest} individuals, is computed using Eq. (15).

$$Pr_i = \frac{R_i}{\sum_{i=1}^{N_{pbest}} R_i} \tag{15}$$

$$R_i = m - \frac{(m-1)\cdot(i-1)}{N_{pbest} - 1} \tag{16}$$

Table 2
Details of MjSO and six MjSO variants.

Strategy	MjSO	MjSO1	MjSO2 (jSO)	MjSO3	MjSO4	MjSO5	MjSO6
Archive scheme	No	Yes	Yes	No	No	Yes	Yes
Probability selection mechanism	Yes	Yes	No	Yes	No	Yes	No
Directed mutation strategy	Yes	Yes	No	No	Yes	No	Yes

Table 3
The rankings of MjSO and its variants derived from the Friedman test (α=0.05).

Algorithms	10D[a]	30D[b]	50D[c]	100D[d]	Mean Ranking	Mean Rank
MjSO	3.52	3.43	3.34	3.28	**3.39**	**1**
MjSO1	4.86	4.78	3.81	3.33	4.19	4
MjSO2(jSO)	3.86	4.26	4.28	4.64	4.26	5
MjSO3	**3.17**	3.84	5.17	5.31	4.38	7
MjSO4	4.14	**3.38**	**3.33**	3.79	3.66	2
MjSO5	4.07	4.29	4.43	4.40	4.30	6
MjSO6	4.38	4.02	3.64	**3.26**	3.82	3

[a] . *p*-value is 1.92E-02, and the *chi-square* is 15.14.
[b] . *p*-value is 1.09E-01, and the *chi-square* is 10.39.
[c] . *p*-value is 4.63E-03, and the *chi-square* is 18.74.
[d] . *p*-value is 5.06E-04, and the *chi-square* is 24.07.

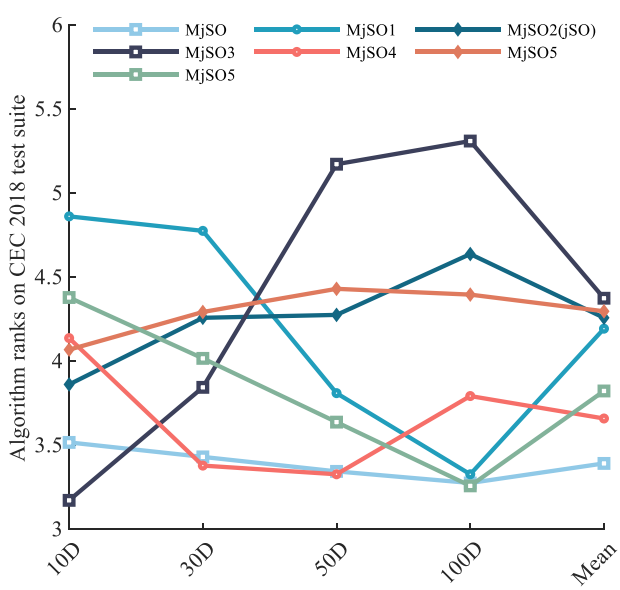


Fig. 4. The rankings of MjSO and its six variants derived from the Friedman test.

Table 4
Parameter settings for MjSO and six comparison algorithms.

Algorithms	Parameter settings
MjSO	H=5, m=6, NP_{min}=4, NP_{init}=25log(D)$\sqrt{D}$, p_{min}=0.125, p_{max}=0.25.
EB-LSHADE	H=5, NP_{min}=4, NP_{init}=18•D, c=0.8, $\|A\|$=1.4•NP, p=0.11 as in Ref. [18].
ELSHADE-SPACMA	H=5, NP_{min}=4, NP_{init}=18•D, $\|A\|$=1.4•NP, c=0.8, F_{CP}=0.5, p_{min}=0.15, p_{max}=0.3, p_{AG}=0.1, $thre$=$FEs_{\max}$/2 as in Ref. [10].
iL-SHADE	H=6, NP_{min}=4, NP_{init}=12•D, $\|A\|$=2.6•NP, $p_{\min}$ =0.1, $p_{\max}$=0.2 as in Ref. [6].
jSO	H=5, NP_{min}=4, NP_{init}=25log(D)$\sqrt{D}$, $\|A\|$=2.6•NP, p_{min}=0.125, p_{max}=0.25 as in Ref. [7].
MadDE	H=10•D, NP_{min}=4, NP_{init}=2•D^2, $\|A\|$=2.3•NP, p_{qBX}= 0.01, Cr_0=0.2, p=0.18, F_0=0.2 as in Ref. [9].
mL-SHADE	H=6, NP_{min}=4, NP_{init}=18•D, $\|A\|$=NP, N^{stuck}=400, p=0.11, η=10, p_m=1/D, m_r=0.05 as in Ref. [16].

where, m is the probability scale factor that is used to adjust the ratio of selected probability between the best dominant individual and the worst dominant individual. By applying Eqs. (15) and (16), all individuals in the domain population, ranked from best to worst, are selected with an arithmetic sequence, and the ratio of selected probability between the best dominant individual and the worst dominant individual remains constant m in the iterative process.

3.2. Directed mutation operator

As described in Section 2.2, differential $\boldsymbol{x}_{r1}$ and $\boldsymbol{x}_{r2}$ are used as part of the mutation strategy. Inspired by the Gaining-Sharing based optimization algorithm (GSK) [33], the novel directed mutation strategy for jSO is proposed in this section to enhance its exploitation. The difference between $\boldsymbol{x}_{r1}$ and $\boldsymbol{x}_{r2}$ is based on its fitness values in the directed mutation strategy to make sure that evolution is going in a better direction. Specifically, the mutated vector v_i for the individual x_i is generated using the directed mutation strategy as Eq. (17) if the problem is a minimization problem.

$$\boldsymbol{v}_i = \begin{cases} \boldsymbol{x}_i + Fw_i \cdot (\boldsymbol{x}_{pbest} - \boldsymbol{x}_i) + F_i \cdot (\boldsymbol{x}_{r1} - \boldsymbol{x}_{r2}), & f(\boldsymbol{x}_{r1}) \le f(\boldsymbol{x}_{r2}) \\ \boldsymbol{x}_i + Fw_i \cdot (\boldsymbol{x}_{pbest} - \boldsymbol{x}_i) + F_i \cdot (\boldsymbol{x}_{r2} - \boldsymbol{x}_{r1}), & f(\boldsymbol{x}_{r1}) > f(\boldsymbol{x}_{r2}) \end{cases} \tag{17}$$

When the problem is a maximization problem, the directed mutation strategy is described as Eq. (18).

$$\boldsymbol{v}_i = \begin{cases} \boldsymbol{x}_i + Fw_i \cdot (\boldsymbol{x}_{pbest} - \boldsymbol{x}_i) + F_i \cdot (\boldsymbol{x}_{r1} - \boldsymbol{x}_{r2}), & f(\boldsymbol{x}_{r1}) \ge f(\boldsymbol{x}_{r2}) \\ \boldsymbol{x}_i + Fw_i \cdot (\boldsymbol{x}_{pbest} - \boldsymbol{x}_i) + F_i \cdot (\boldsymbol{x}_{r2} - \boldsymbol{x}_{r1}), & f(\boldsymbol{x}_{r1}) < f(\boldsymbol{x}_{r2}) \end{cases} \tag{18}$$

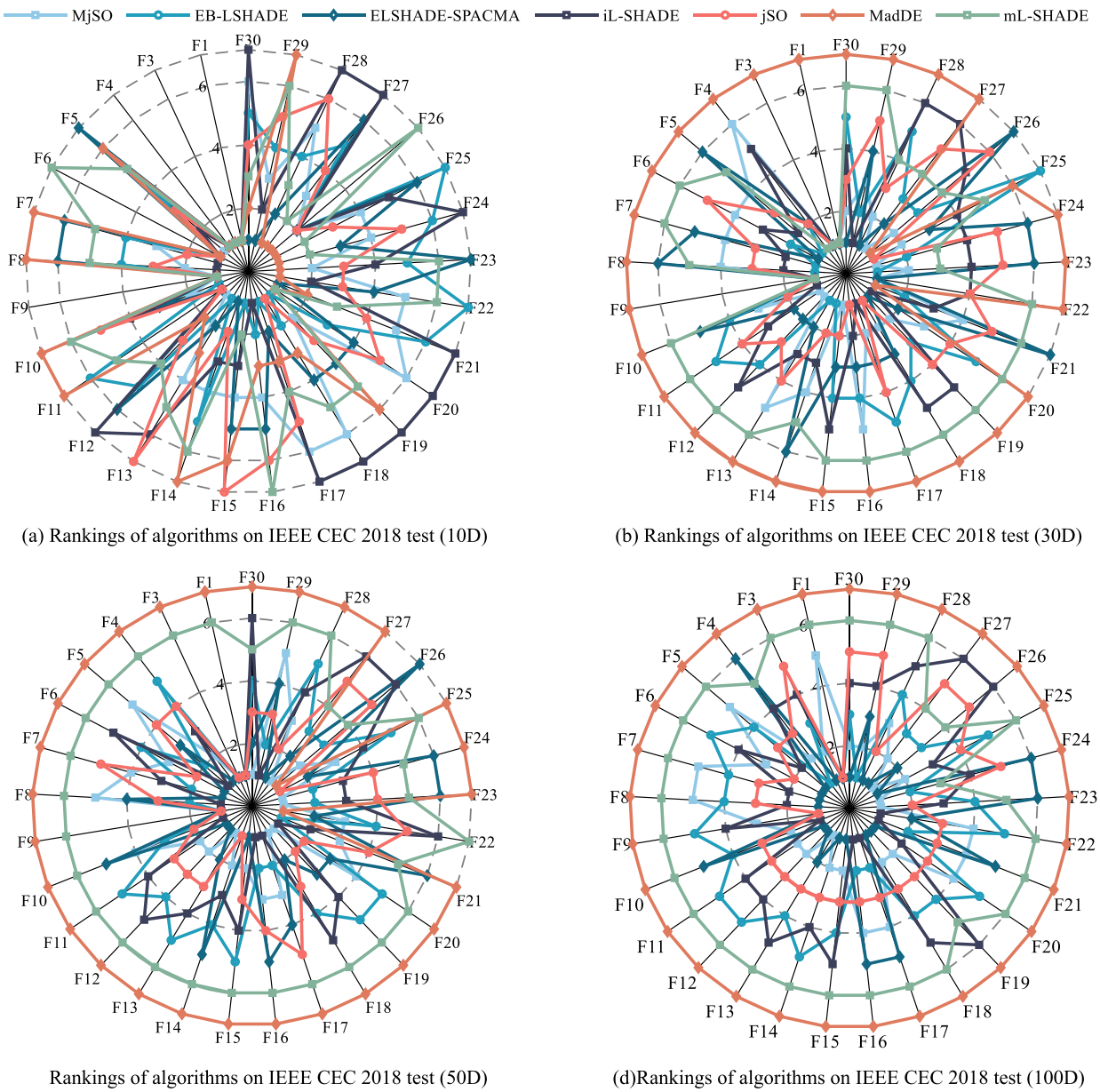
The illustration of the directed mutation operator in a two-dimensional space is presented in Fig. 1, in which $\boldsymbol{x}_{r2}$ is better than $\boldsymbol{x}_{r1}$. It is obvious that the mutated vector v_i generated using the directed mutation operator is better than that generated using the original mutation operator.

What is more, the archive scheme is no longer adopted in MjSO to avoid excessive use of historically dominant individuals. Thus, $\boldsymbol{x}_{r2}$ is randomly selected from the current population of MjSO. The pseudo-code of MjSO is presented in Fig. 2.

4. Numerical experiments based on IEEE CEC 2018 test suite

This section is devoted to investigating the performance of MjSO based on the IEEE CEC 2018 test suite [34] on single objective bound constrained real-parameter optimization. In the CEC 2018 test suite, F01 and F03 are unimodal functions, F04-F10 are multimodal functions, F11-F20 are hybrid functions, and F21-F30 are composition functions, and more details about these functions are available in Ref. [34]. According to the guideline in Ref. [34], the search range is $[-100,100]^D$ and the $FEs_{\max}$ is set to $D \times 10{,}000$ for all benchmark functions included in the IEEE CEC 2018 test suite. D is set to 10, 30, 50, and 100. Each algorithm is independently run 51 times. The error measure $f(\boldsymbol{x}_{Best})$-$f(\boldsymbol{x}^*)$ is recorded as the result, where $\boldsymbol{x}_{Best}$ is the solution obtained by one algorithm, and $\boldsymbol{x}^*$ denotes the global optimum solution of the benchmark function [35]. The error measure will be taken as 0 if it is smaller than 10^{-8}. All experiments are implemented using MATLAB R2019b on a computer with Windows 10, i7-10750H 2.6 GHz CPU, and 16GB RAM.

To avoid inaccurate conclusions caused by statistical averages of experimental results, two non-parametric statistical hypothesis tests, the



(a) Rankings of algorithms on IEEE CEC 2018 test (10D)

(b) Rankings of algorithms on IEEE CEC 2018 test (30D)

Rankings of algorithms on IEEE CEC 2018 test (50D)

(d)Rankings of algorithms on IEEE CEC 2018 test (100D)

Fig. 5. Rankings based on "Mean" of MjSO and six comparison algorithms on each benchmark function.

Table 5
Results derived from the Wilcoxon signed-rank test (α=0.05).

R (+/-/=)	10D	30D	50D	100D	Total
MjSO vs. jSO	4/0/25	8/1/20	10/4/15	12/6/11	34/11/71
MjSO vs. ELSHADE-SPACMA	8/6/15	14/2/13	10/6/13	6/16/7	38/30/48
MjSO vs. EB-LSHADE	8/4/17	11/6/12	16/5/8	17/6/6	52/21/43
MjSO vs. iL-SHADE	6/5/18	11/7/11	14/8/7	14/9/6	45/29/42
MjSO vs. MadDE	10/7/12	27/2/0	27/2/0	29/0/0	93/11/12
MjSO vs. mL-SHADE	12/2/15	25/1/3	28/0/1	27/0/2	92/3/21

Table 6
Rankings of MjSO and six comparison algorithms derived from the Friedman test (α=0.05).

Algorithms	10D[a]	30D[b]	50D[c]	100D[d]	Mean Ranking	Rank
MjSO	3.72	**2.84**	**2.62**	2.72	**2.98**	**1**
EB-LSHADE	3.88	3.12	3.41	3.60	3.50	4
ELSHADE-SPACMA	3.86	3.53	3.07	**2.31**	3.19	2
iL-SHADE	4.48	3.45	3.31	3.43	3.67	5
jSO	3.97	3.48	3.24	3.28	3.49	3
MadDE	**3.66**	6.55	6.59	7.00	5.95	7
mL-SHADE	4.43	5.02	5.76	5.66	5.22	6

[a] . *p*-value is 5.56E-01, and the *chi-square* is 4.91.
[b] . *p*-value is 4.51E-13, and the *chi-square* is 69.79.
[c] . *p*-value is 3.05E-17, and the *chi-square* is 89.98.
[d] . *p*-value is 5.92E-21, and the *chi-square* is 107.79.

Wilcoxon signed-rank test and the Friedman test [36], are used to analyze the experimental results in this paper. Specifically, the Wilcoxon signed-rank test is utilized to judge which algorithm has better performance for each function, and the rankings of algorithms could be obtained by the Friedman test to test significant differences among

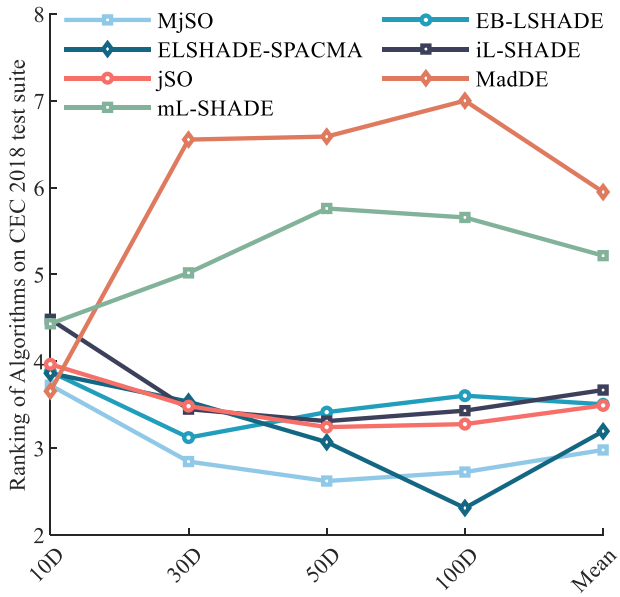


Fig. 6. Rankings of MjSO and six comparison algorithms derived from the Friedman test (α=0.05).

algorithms [32].

4.1. MjSO parametric analysis

In the study of evolutionary algorithms, the tuning of parameters plays a substantial role in influencing algorithms performance. To give full play to the performance of MjSO, the probability scale control factor m, one new parameter introduced into MjSO, needs to be discussed to obtain the optimal setting. The other parameters settings in MjSO are consistent with that in jSO.

To avoid parameter overfitting, six benchmark functions selected from the CEC 2018 test suite instead of all benchmarks are employed to assess the performance of MjSO with different values of parameter m. Note that MjSO is independently run 30 times to solve each benchmark function in this section. The results obtained by MjSO with different values of parameter m are tabulated in Table 1. The rankings derived from the Friedman test (F-Ranks) of MjSO with different values of parameter m are also listed in Table 1 and visually shown as Fig. 3. From Fig. 3, the best parameter settings occur at m=6, where the statistical ranking value is the best at 6.67. Therefore, the probability scale factor m is set to 6 to fully display the performance of MjSO in this paper.

4.2. Strategies effectiveness analysis

The effectiveness of modifications proposed in our work is investigated using the CEC 2018 test suite. Six MjSO variants, namely MjSO1, MjSO2, MjSO3, MjSO4, MjSO5, and MjSO6, are designed in this section to conduct effectiveness analysis of the modifications. Details of MjSO and its variants are as shown in Table 2. It is noted that the reason why MjSO2 is the original jSO is that it uses only the archive scheme. The experimental statistical results are provided in Tables S1–S4 included in the supplementary file.

For an overall comparison in a statistical manner between MjSO and its variants, the Friedman test is utilized to analyze the experimental results listed in Tables S1–S4. The rankings of MjSO and its six variants derived from the Friedman test are summarized in Table 3. All p-values except p-value for 30D derived from the Friedman test are not more than α, i.e., there are significant differences between the performance of MjSO, MjSO1, MjSO2, MjSO3, MjSO4, MjSO5, and MjSO6. The rankings of MjSO and its variants are visually shown in Fig. 4.

From Fig. 4, MjSO ranks in the top in the "Mean Ranking" which means MjSO is better than its variants in terms of overall performance, even though MjSO is worse than MjSO3 in 10D, MjSO4 in 30D and 50D, and MjSO6 in 100D. The detailed analysis is as follows. MjSO outperforms MjSO1 on all dimensions indicating that the archive scheme weakens the performance of MjSO. Therefore, the archive scheme is no longer adopted in the proposed MjSO in this paper. MjSO outperforms MjSO3 on 30D, 50D, and 100D indicating that the directed mutation strategy effectively enhances the performance of MjSO, even if MjSO is slightly inferior to MjSO3 on 10D. MjSO is significantly better than MjSO4 on 10D and 100D, and has a similar performance to MjSO4 on 30D and 50D indicates that the probability selection mechanism effectively enhances the performance of MjSO. MjSO outperforms MjSO2 (jSO) on all dimensions indicating that combining the probability selection mechanism and the directed mutation strategy effectively enhance jSO. In summary, it can be concluded that the probability selection mechanism and the directed mutation strategy can effectively enhance jSO.

4.3. Comparison with state-of-the-art algorithms

MjSO is evaluated using the CEC 2018 test suite in comparison with six DE variants in this section. The six state-of-the-art DE variants include EB-LSHADE [18], ELSHADE-SPACMA [10], iL-SHADE [6], jSO [7], MadDE [9], and mL-SHADE [16]. The parameters of the comparison algorithms are tabulated in Table 4 and those parameters are set as suggested in the original paper for a fair comparison. The detailed statistical results obtained by MjSO, ELSHADE-SPACMA, EB-LSHADE, iL-SHADE, jSO, MadDE, and mL-SHADE are illustrated in Tables S5–S8 that can be found in the supplementary file.

To show the performance only based on the mean value more

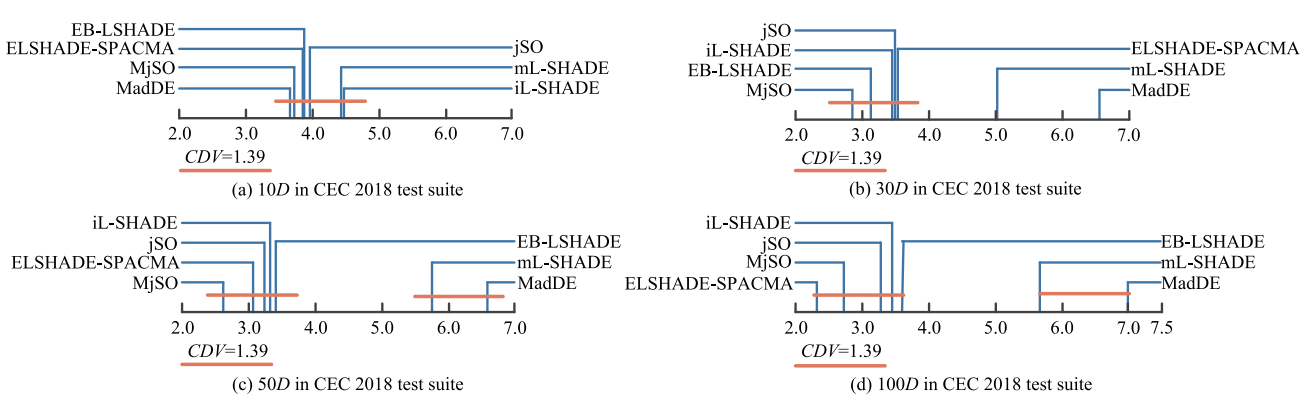


Fig. 7. The differences among MjSO and six comparison algorithms.

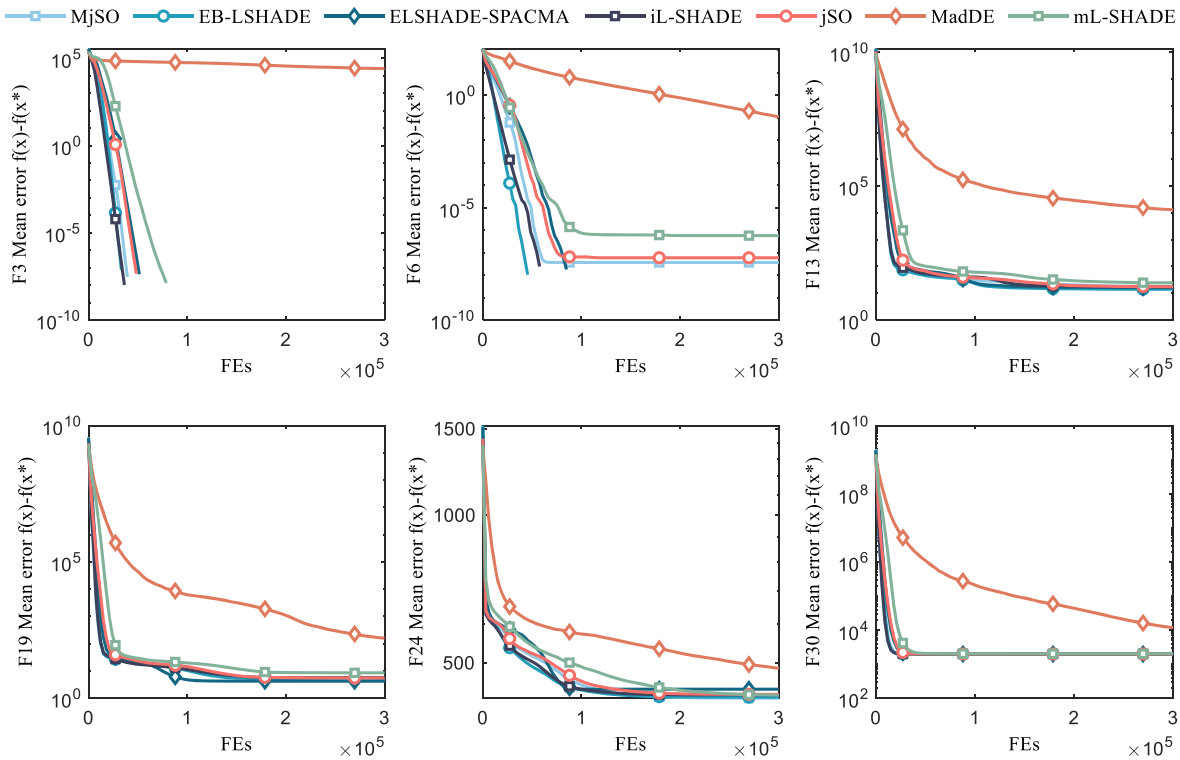


Fig. 8. Average convergence curves of MjSO and six comparison algorithms on six functions (D=30).

Table 7
Computational complexity of MjSO and six comparison algorithms.

Algorithms	$\bar{T}_2$/s				$(\bar{T}_2 - T_1)/T_0$			
	10D	30D	50D	100D	10D	30D	50D	100D
MjSO	0.2656	0.6077	1.0686	2.8675	**18.0061**	29.7710	41.9784	**68.0132**
EB-LSHADE	0.8882	1.2543	1.8081	3.9783	76.2260	90.2324	111.1259	171.8761
ELSHADE-SPACMA	0.6620	1.0863	1.7869	4.6642	55.0763	74.5251	109.1471	236.0063
iL-SHADE	0.3079	0.5970	1.0489	2.9635	21.9649	**28.7670**	**40.1397**	76.9875
jSO	0.6164	0.9219	1.4206	3.3531	50.8121	59.1464	74.8927	113.4197
MadDE	0.6660	1.1248	1.7133	3.9179	55.4486	78.1177	102.2579	166.2211
mL-SHADE	0.5405	0.8496	1.2569	3.0069	43.7087	52.3850	59.5930	81.0467

T_0=0.0107 s, T_1=0.0730 s (10D), T_1=0.2893s (30D), T_1=0.6196 s (50D). T_1=2.1401s (100D).

intuitively in each benchmark function, the spider charts based on rankings of the "Mean" in Tables S5–S8 are presented in Fig. 5. In Fig. 5, it can be roughly concluded that MjSO is obviously better than all competitors on all dimensions. In the following subsections, statistical methods are used to accurately analyze the experimental results to avoid inaccurate conclusions caused by statistical averages of experimental results.

4.3.1. Analysis using the Wilcoxon signed-rank test

In this section, the Wilcoxon signed-rank test is utilized to analyze the results obtained by MjSO and six comparison algorithms running 51 times. The Wilcoxon signed rank test (α=0.05) results are labeled "R" in Tables S5–S8 and summarized in Table 5, in which '+' denotes that MjSO outperforms the competitor, whereas '−' denotes that MjSO underperforms the competitor, and '=' denotes that MjSO is statistically like the competitor.

According to the "Total" in Table 5, there are higher total of '+' than '-' by comparing MjSO with jSO, ELSHADE-SPACMA, EB-LSHADE, iL-SHADE, mL-SHADE, and MadDE. That means MjSO outperforms six comparison algorithms in terms of overall performance. Details of the analysis are given below.

a) On 10D, MjSO outperforms (underperforms) jSO on 4(0) benchmark functions, ELSHADE-SPACMA on 8(6) benchmark functions, EB-LSHADE on 8(4) benchmark functions, iL-SHADE on 6(5) benchmark functions, MadDE on 10(7) benchmark functions, and mL-SHADE on 12(2) benchmark functions. It can be concluded that MjSO outperforms six comparison algorithms on 10D.
b) On 30D, MjSO outperforms (underperforms) jSO on 8(1) benchmark functions, ELSHADE-SPACMA on 14(2) benchmark functions, EB-LSHADE on 11(6) benchmark functions, iL-SHADE on 11(7) benchmark functions, MadDE on 27(2) benchmark functions, and mL-SHADE on 25(1) benchmark functions. It can be concluded that MjSO outperforms six comparison algorithms on 30D.
c) On 50D, MjSO outperforms (underperforms) jSO on 10(4) benchmark functions, ELSHADE-SPACMA on 10(6) benchmark functions, EB-LSHADE on 16(5) benchmark functions, iL-SHADE on 14(8) benchmark functions, MadDE on 27(2) benchmark functions, and mL-SHADE on 28(0) benchmark functions. It can be concluded that MjSO outperforms six comparison algorithms on 50D.
d) On 100D, MjSO outperforms (underperforms) jSO on 12(6) benchmark functions, ELSHADE-SPACMA on 6(16) benchmark functions, EB-LSHADE on 17(6) benchmark functions, iL-SHADE on 14(9) benchmark functions, MadDE on 29(0) benchmark functions, and

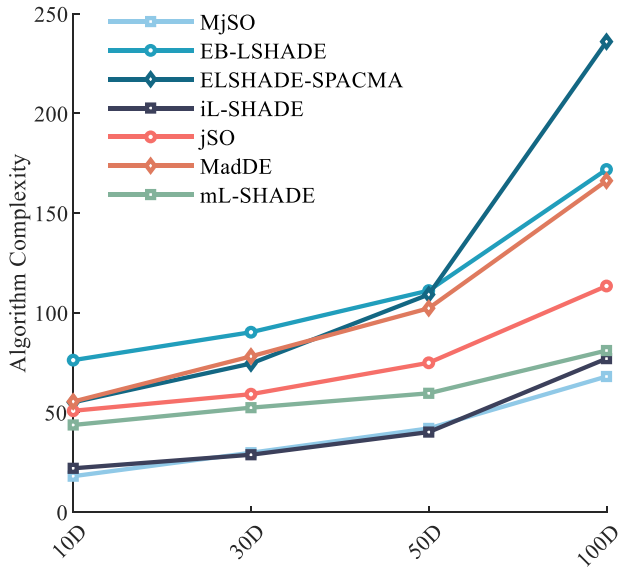


Fig. 9. Computational complexity of MjSO and six comparison algorithms.

mL-SHADE on 27(0) benchmark functions. It can be concluded that MjSO outperforms jSO, mL-SHADE, EB-LSHADE, iL-SHADE, and MadDE, but underperforms ELSHADE-SPACMA on 100D.

4.3.2. Analysis using the Friedman test

In this section, the statistical analysis using the Friedman test (α=0.05) for the "Mean" listed in Tables S5–S8 is carried out to make an overall comparison among MjSO and six comparison algorithms. The Friedman test results are summarized in Table 6. *p*-values for all dimensions except 10D derived from the Friedman test are not more than α, i.e., there are significant differences among the performance of MjSO and six comparison algorithms. The rankings listed in Table 6 are visually shown as Fig. 6.

For 10D, MjSO ranks in second place following in MadDE, followed by ELSHADE-SPACMA, EB-LSHADE, jSO, mL-SHADE, and iL-SHADE. For 30D, MjSO ranks first followed by EB-LSHADE, iL-SHADE, jSO, ELSHADE-SPACMA, mL-SHADE, and MadDE. For 50D, MjSO ranks in the first place followed by ELSHADE-SPACMA, jSO, iL-SHADE, EB-LSHADE, mL-SHADE, and MadDE. For 100D, MjSO ranks in second place following in ELSHADE-SPACMA, followed by jSO, iL-SHADE, EB-LSHADE, mL-SHADE, and MadDE. According to the 'Mean Ranking', MjSO ranks in first place followed by ELSHADE-SPACMA, jSO, EB-LSHADE, iL-SHADE, mL-SHADE, and MadDE. It can be concluded that MjSO is an excellent algorithm and superior to all competitors used in this paper.

Furthermore, the Iman-Davenport test and the Nemenyi test adopted in our previous work [32] are also used as the post hoc test based on the Friedman test results to further analyze the magnitude of significant differences. The post hoc test results are presented Fig. 7. From Fig. 7, MjSO ranks in the second place and has no significant differences with the six comparison algorithms on 10D. As for 30D, 50D, and 100D, MjSO is significantly superior to mL-SHADE and MadDE, but has no significant differences with ELSHADE-SPACMA, jSO, EB-LSHADE, and iL-SHADE. In conclusion, MjSO further enhances the performance of jSO, even if jSO is a very competitive algorithm.

4.3.3. Analysis of convergence rate

The average convergence curve of 51 independent results obtained by MjSO and six comparison algorithms is shown in Fig. 8. For brevity, only six benchmark functions on 30D are used in this section, and the average convergence curves for all functions on all dimensions are depicted in Figs. S1–S8 provided in the supplementary file. It is noted that the average value is taken as 0 if it is less than 1e-08, and the values that are equal to 0 will be ignored in the convergence graphs because they are plotted with a base of 10 logarithmic scales for the y-axis. The convergence speed of MjSO is like that of ELSHADE-SPACMA, EB-LSHADE, iL-SHADE, mL-SHADE, and jSO, and is faster than that of MadDE on the six sampled functions. Therefore, MjSO provides a similar performance to jSO, ELSHADE-SPACMA, EB-LSHADE, mL-SHADE, and iL-SHADE in terms of convergence speed.

4.4. Analysis of computational complexity

In this section, the instruction for evaluating algorithms complexity in the IEEE CEC 2018 test suite [34] is adopted to evaluate the algorithm complexity of MjSO by comparing it with EB-LSHADE, ELSHADE-SPACMA, iL-SHADE, jSO, MadDE, and mL-SHADE. The computational complexity is calculated using $(\bar{T}_2 - T_1)/T_0$, and the meanings of T_0, T_1, and $\bar{T}_2$ can refer to Ref. [34]. In this section, NP_{init} is set to 500 for MjSO and six comparison algorithms on all dimensions, and the other parameters are set as tabulated in Table 4 for fair and rational comparison.

The computational complexity of MjSO, jSO, ELSHADE-SPACMA, EB-LSHADE, iL-SHADE, MadDE, and mL-SHADE is tabulated as Table 7 and visually depicted in Fig. 9. The computational complexity of MjSO is lower than that of jSO, mL-SHADE, ELSHADE-SPACMA, EB-LSHADE, and MadDE on all dimensions. However, the computational complexity of MjSO is slightly higher than that of iL-SHADE on 30D and 50D. The main reason why the computational complexity of MjSO is lower than that of jSO is that the archive scheme is no longer adopted in MjSO. In summary, MjSO reduces the algorithm complexity while improving optimization performance, and it is an efficient algorithm.

5. Conclusion and future work

The paper proposes the MjSO, a novel modified version of jSO, by designing a probability selection mechanism and a directing mutation strategy to further enhance its performance. The CEC 2018 test suite is adopted to verify the performance of MjSO by comparing with jSO and five state-of-the-art DE variants. The experimental results indicate that MjSO outperforms jSO, ELSHADE-SPACMA, EB-LSHADE, iL-SHADE, MadDE, and mL-SHADE. Moreover, MjSO significantly enhances the performance of jSO while reducing its computational complexity, which means that MjSO can achieve more precise results in less time when it is applied to real-world problems. In conclusion, the probability selection mechanism and directed mutation strategy proposed in this study efficiently enhance the performance of jSO, and MjSO is an excellent algorithm.

The parameter tunes only based on one test suite is the limitation of our work. There are three directions for future work. Firstly, the parameters of MjSO can be tuned more accurately to fully develop its performance. Secondly, the idea of directed mutation strategy can be migrated to other heuristic algorithms to promote the development of evolutionary algorithms. Finally, MjSO can be used to solve the dynamic target assignment of UAV swarm.

Author agreement

The authors claim that none of the material in the paper has been published or is under consideration for publication elsewhere, and no conflict of interest exits in the submission of this manuscript, and the manuscript is approved by all authors for publication.

CRediT authorship contribution statement

Yintong Li: Conceptualization, Methodology, Data curation, Investigation, Writing – original draft, Writing – review & editing. **Tong Han:** Conceptualization, Data curation, Investigation. **Xiaofei Wang:**

Conceptualization, Software, Validation, Writing – review & editing. **Huan Zhou:** Software, Writing – review & editing, Funding acquisition. **Shangqin Tang:** Investigation, Writing – review & editing. **Changqiang Huang:** Supervision, Project administration, Writing – review & editing. **Bo Han:** Validation, Writing – review & editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the Natural Science Foundation of Shaanxi Province [grant numbers 2021JM223, 2021JM224], the National Natural Science Foundation of China [grant number 62101590].

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.swevo.2023.101294.

References

[1] S. Das, S.S. Mullick, P.N. Suganthan, Recent advances in differential evolution – an updated survey, Swarm Evol. Comput. 27 (2016) 1–30, https://doi.org/10.1016/j.swevo.2016.01.004.
[2] R. Tanabe, A.S. Fukunaga, Improving the search performance of SHADE using linear population size reduction, in: Proceedings of the 2014 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2014, pp. 1658–1665, https://doi.org/10.1109/CEC.2014.6900380.
[3] K.R. Opara, J. Arabas, Differential evolution: a survey of theoretical analyses, Swarm Evol. Comput. 44 (2019) 546–558, https://doi.org/10.1016/j.swevo.2018.06.010.
[4] M. Hamdi, L. Idomghar, M. Chaoui, A. Kachouri, An improved adaptive differential evolution optimizer for non-convex economic dispatch problems, Appl. Soft Comput. 85 (2019), 105868, https://doi.org/10.1016/j.asoc.2019.105868.
[5] N.H. Awad, M.Z. Ali, P.N. Suganthan, R.G. Reynolds, An ensemble sinusoidal parameter adaptation incorporated with L-SHADE for solving CEC2014 benchmark problems, in: Proceedings of the 2016 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2016, pp. 2958–2965, https://doi.org/10.1109/CEC.2016.7744163.
[6] J. Brest, M.S. Maučec, B. Bošković, IL-SHADE: improved L-SHADE algorithm for single objective real-parameter optimization, in: Proceedings of the 2016 IEEE Congress on Evolutionary Computation, CEC, 2016, pp. 1188–1195, https://doi.org/10.1109/CEC.2016.7743922, 2016.
[7] J. Brest, M.S. Maucec, B. Boskovic, Single objective real-parameter optimization: algorithm jSO, in: Proceedings of the 2017 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2017, pp. 1311–1318, https://doi.org/10.1109/CEC.2017.7969456.
[8] J. Sun, X. Liu, T. Back, Z. Xu, Learning adaptive differential evolution algorithm from optimization experiences by policy gradient, IEEE Trans. Evol. Comput. 25 (2021) 666–680, https://doi.org/10.1109/TEVC.2021.3060811.
[9] S. Biswas, D. Saha, S. De, A.D. Cobb, S. Das, B.A. Jalaian, Improving differential evolution through Bayesian hyperparameter optimization, in: Proceedings of the 2021 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2021, pp. 832–840, https://doi.org/10.1109/CEC45853.2021.9504792.
[10] A.A. Hadi, A.W. Mohamed, K.M. Jambi, Single-objective real-parameter optimization: enhanced LSHADE-SPACMA Algorithm, in: Proceedings of the Studies in Computational Intelligence, 2021, pp. 103–121, https://doi.org/10.1007/978-3-030-58930-1_7.
[11] A.A. Hadi, A.W. Mohamed, K.M. Jambi, Single-objective real-parameter optimization : enhanced LSHADE-SPACMA algorithm, 2018, doi:10.13140/RG.2.2.33283.20005. https://www.researchgate.net/publication/326231075.
[12] V. Stanovov, S. Akhmedova, E. Semenkin, LSHADE algorithm with rank-based selective pressure strategy for solving CEC 2017 benchmark problems, in: Proceedings of the 2018 IEEE Congress on Evolutionary Computation, CEC 2018, IEEE, 2018, pp. 1–8, https://doi.org/10.1109/CEC.2018.8477977.
[13] K. Jebari, M. Madiafi, Selection methods for genetic algorithms, Int. J. Emerg. Sci. 3 (2013) 333–344.
[14] T.J. Choi, C.W. Ahn, An improved LSHADE-RSP algorithm with the Cauchy perturbation: iLSHADE-RSP, Knowl. Based Syst. 215 (2021), 106628, https://doi.org/10.1016/j.knosys.2020.106628.
[15] A.P. Piotrowski, L-SHADE optimization algorithms with population-wide inertia, Inf. Sci. (NY) 468 (2018) 117–141, https://doi.org/10.1016/j.ins.2018.08.030.
[16] J.F. Yeh, T.Y. Chen, T.C. Chiang, Modified L-SHADE for single objective real-parameter optimization, in: Proceedings of the 2019 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2019, pp. 381–386, https://doi.org/10.1109/CEC.2019.8789991.
[17] W. Yi, Y. Chen, Z. Pei, J. Lu, Adaptive differential evolution with ensembling operators for continuous optimization problems, Swarm Evol. Comput. 69 (2022), 100994, https://doi.org/10.1016/j.swevo.2021.100994.
[18] A.W. Mohamed, A.A. Hadi, K.M. Jambi, Novel mutation strategy for enhancing SHADE and LSHADE algorithms for global numerical optimization, Swarm Evol. Comput. 50 (2019), 100455, https://doi.org/10.1016/j.swevo.2018.10.006.
[19] A.W. Mohamed, A.A. Hadi, A.K. Mohamed, Differential evolution mutations: taxonomy, comparison and convergence analysis, IEEE Access 9 (2021) 68629–68662, https://doi.org/10.1109/ACCESS.2021.3077242.
[20] N.H. Awad, M.Z. Ali, P.N. Suganthan, Ensemble sinusoidal differential covariance matrix adaptation with Euclidean neighborhood for solving CEC2017 benchmark problems, in: Proceedings of the 2017 IEEE Congress on Evolutionary Computation, CEC 2017, 2017, pp. 372–379, https://doi.org/10.1109/CEC.2017.7969336.
[21] A. Kumar, P.P. Biswas, P.N. Suganthan, Differential evolution with orthogonal array-based initialization and a novel selection strategy, Swarm Evol. Comput. 68 (2022), 101010, https://doi.org/10.1016/j.swevo.2021.101010.
[22] Z. Zeng, M. Zhang, T. Chen, Z. Hong, A new selection operator for differential evolution algorithm, Knowl. Based Syst. 226 (2021), 107150, https://doi.org/10.1016/j.knosys.2021.107150.
[23] Z. Zeng, Z. Hong, H. Zhang, M. Zhang, C. Chen, Improving differential evolution using a best discarded vector selection strategy, Inf. Sci. (NY) 609 (2022) 353–375, https://doi.org/10.1016/j.ins.2022.07.075.
[24] A.W. Mohamed, A.A. Hadi, A.M. Fattouh, K.M. Jambi, LSHADE with semi-parameter adaptation hybrid with CMA-ES for solving CEC 2017 benchmark problems, in: Proceedings of the 2017 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2017, pp. 145–152, https://doi.org/10.1109/CEC.2017.7969307.
[25] F. Zhao, H. Bao, L. Wang, X. He, Jonrinaldi, A hybrid cooperative differential evolution assisted by CMA-ES with local search mechanism, Neural Comput. Appl. 34 (2022) 7173–7197, https://doi.org/10.1007/s00521-021-06849-z.
[26] Z. Meng, C. Yang, Hip-DE: historical population based mutation strategy in differential evolution with parameter adaptive mechanism, Inf. Sci. 562 (2021) 44–77, https://doi.org/10.1016/j.ins.2021.01.031.
[27] Z. Tan, K. Li, Y. Wang, Differential evolution with adaptive mutation strategy based on fitness landscape analysis, Inf. Sci. 549 (2021) 142–163, https://doi.org/10.1016/j.ins.2020.11.023.
[28] V. Stanovov, S. Akhmedova, E. Semenkin, Biased parameter adaptation in differential evolution, Inf. Sci. 566 (2021) 215–238, https://doi.org/10.1016/j.ins.2021.03.016.
[29] J. Brest, M.S. Maucec, B. Boskovic, Self-adaptive differential evolution algorithm with population size reduction for single objective bound-constrained optimization: algorithm j21, in: Proceedings of the 2021 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2021, pp. 817–824, https://doi.org/10.1109/CEC45853.2021.9504782.
[30] K.B. Cantún-Avila, D. González-Sánchez, S. Díaz-Infante, F. Peñuñuri, Optimizing functionals using differential evolution, Eng. Appl. Artif. Intell. 97 (2021), 104086, https://doi.org/10.1016/j.engappai.2020.104086.
[31] F. Tao, L. Jiang, C. Li, Differential evolution-based weighted soft majority voting for crowdsourcing, Eng. Appl. Artif. Intell. 106 (2021), 104474, https://doi.org/10.1016/j.engappai.2021.104474.
[32] Y. Li, T. Han, H. Zhou, S. Tang, H. Zhao, A novel adaptive L-SHADE algorithm and its application in UAV swarm resource configuration problem, Inf. Sci. 606 (2022) 350–367, https://doi.org/10.1016/j.ins.2022.05.058.
[33] A.W. Mohamed, A.A. Hadi, A.K. Mohamed, Gaining-sharing knowledge based algorithm for solving optimization problems: a novel nature-inspired algorithm, Int. J. Mach. Learn. Cybern. 11 (2020) 1501–1529, https://doi.org/10.1007/s13042-019-01053-x.
[34] N.H. Awad, M.Z. Ali, J. Liang, B.Y. Qu, P.N. Suganthan, Problem definitions and evaluation criteria for the CEC 2017 special session and competition on real-parameter optimization. https://github.com/P-N-Suganthan/CEC2017-BoundContrained.
[35] Y. Li, T. Han, S. Tang, C. Huang, H. Zhou, Y. Wang, An improved differential evolution by hybridizing with estimation-of-distribution algorithm, Inf. Sci. (NY) 619 (2023) 439–456, https://doi.org/10.1016/j.ins.2022.11.029.
[36] M. Hollander, D.A. Wolfe, E. Chicken, Nonparametric Statistical Methods, Wiley, 2015, https://doi.org/10.1002/9781119196037.